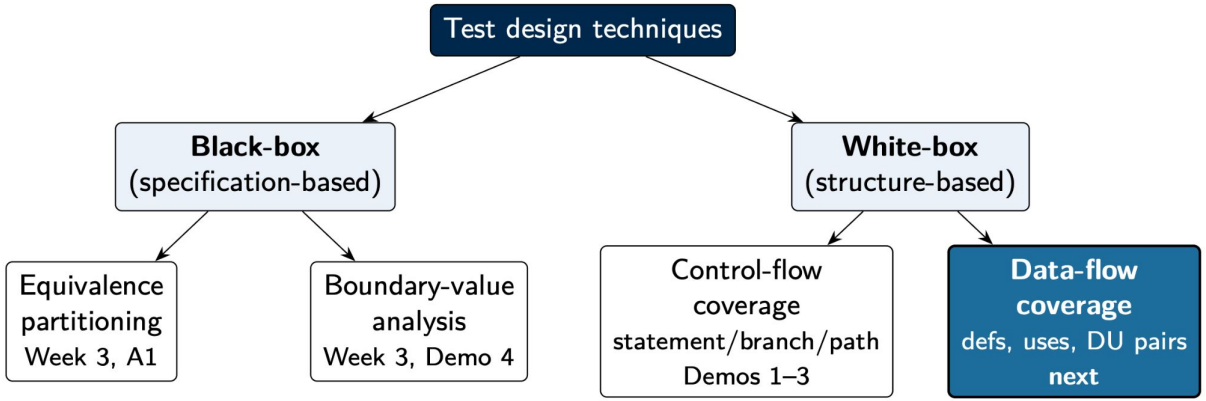


SE 413/513 (Software Testing)

GitHub Repository

Control Flow Testing

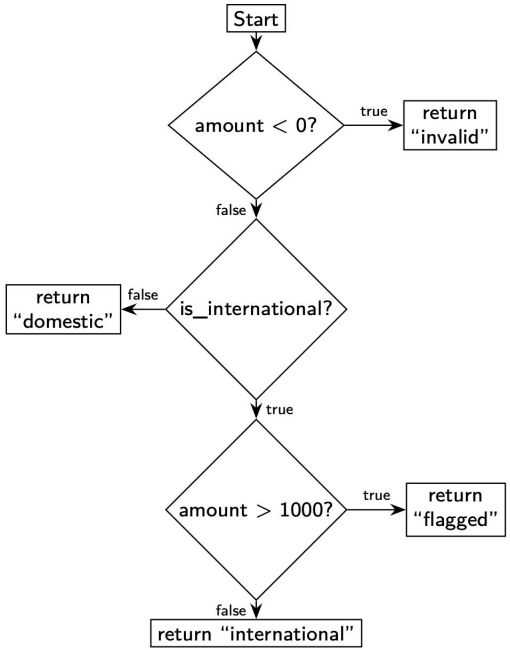


Control-Flow Graph

app/payments/transactions.py

```
def classify_transaction(amount,
     is_international):
    if amount < 0:
        return "invalid"
    if is_international:
        if amount > 1000:
            return "flagged"
        else:
            return "international"
    else:
        return "domestic"
```

Nb Statements: 7



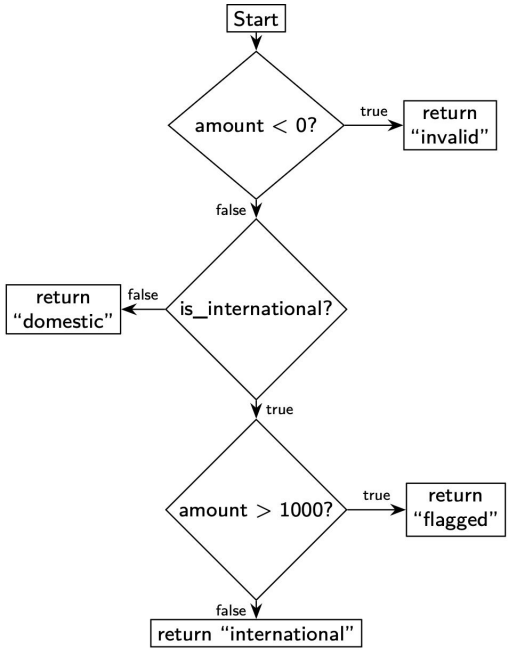
Control-Flow Graph

app/payments/transactions.py

```
def classify_transaction(amount,
     is_international):
    if amount < 0:
        return "invalid"
    if is_international:
        if amount > 1000:
            return "flagged"
        else:
            return "international"
    else:
        return "domestic"
```

Nb Statements: 7

Nb Branches : 2 + 2 + 2



Control-Flow Graph

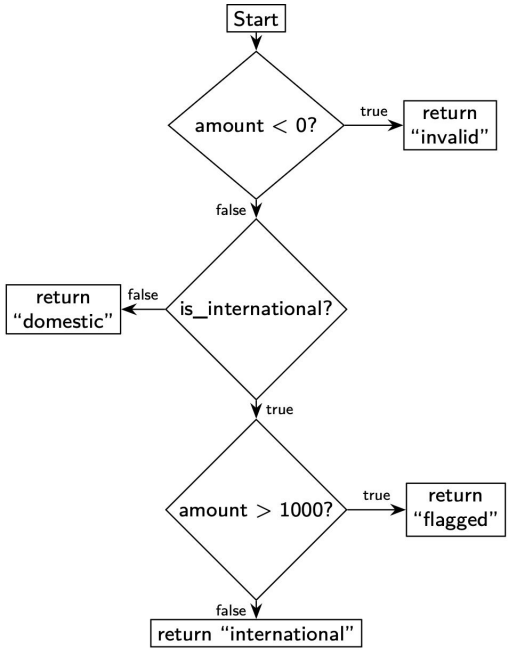
app/payments/transactions.py

```
def classify_transaction(amount,
     is_international):
    if amount < 0:
        return "invalid"
    if is_international:
        if amount > 1000:
            return "flagged"
        else:
            return "international"
    else:
        return "domestic"
```

Nb Statements: 7

Nb Branches : 2 + 2 + 2

Nb Paths: ??



Control-Flow Graph

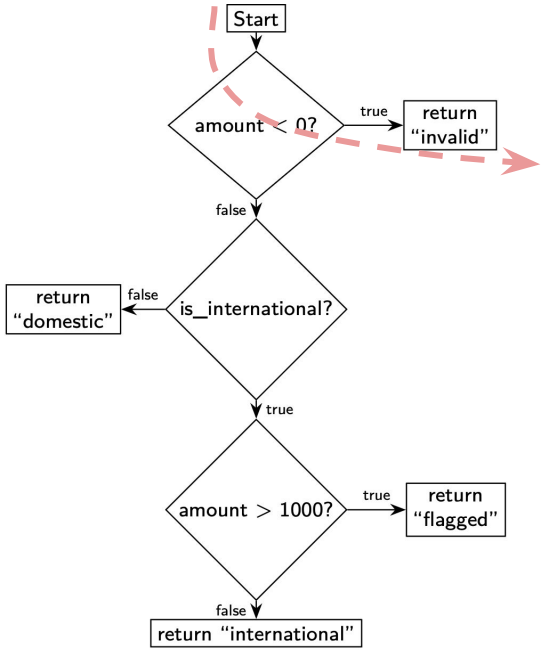
app/payments/transactions.py

```
def classify_transaction(amount,
     is_international):
    if amount < 0:
        return "invalid"
    if is_international:
        if amount > 1000:
            return "flagged"
        else:
            return "international"
    else:
        return "domestic"
```

Nb Statements: 7

Nb Branches : 2 + 2 + 2

Nb Paths: ??



Control-Flow Graph

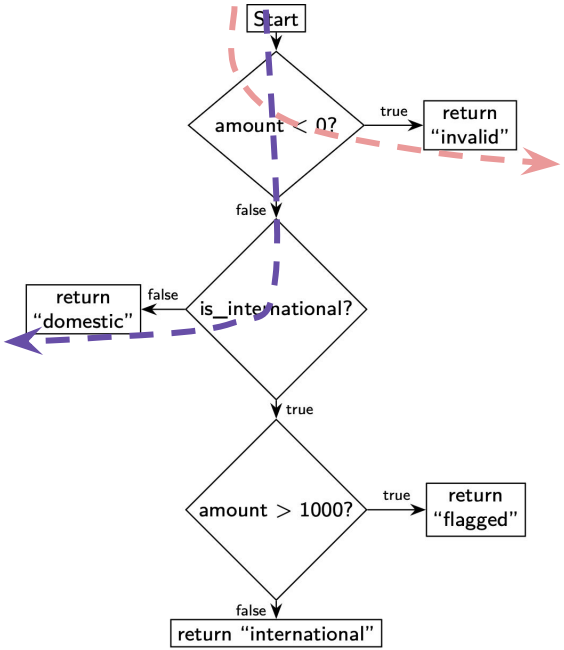
app/payments/transactions.py

```
def classify_transaction(amount,
     is_international):
    if amount < 0:
        return "invalid"
    if is_international:
        if amount > 1000:
            return "flagged"
        else:
            return "international"
    else:
        return "domestic"
```

Nb Statements: 7

Nb Branches : 2 + 2 + 2

Nb Paths: ??



Control-Flow Graph

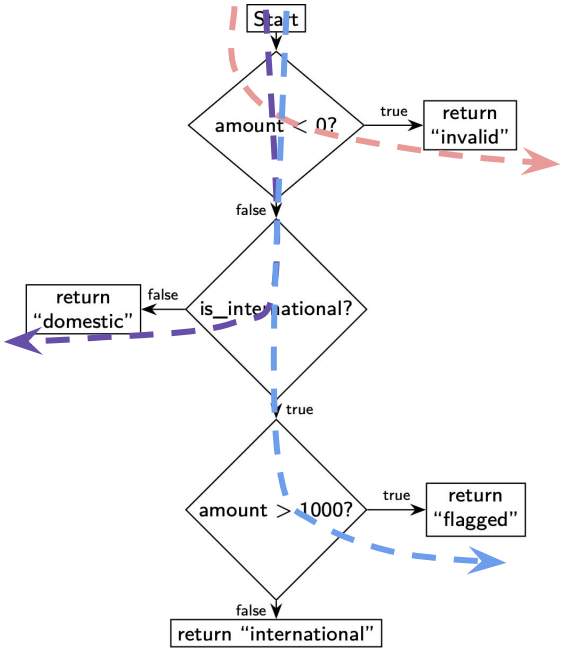
app/payments/transactions.py

```
def classify_transaction(amount,
     is_international):
    if amount < 0:
        return "invalid"
    if is_international:
        if amount > 1000:
            return "flagged"
        else:
            return "international"
    else:
        return "domestic"
```

Nb Statements: 7

Nb Branches : 2 + 2 + 2

Nb Paths: ??



Control-Flow Graph

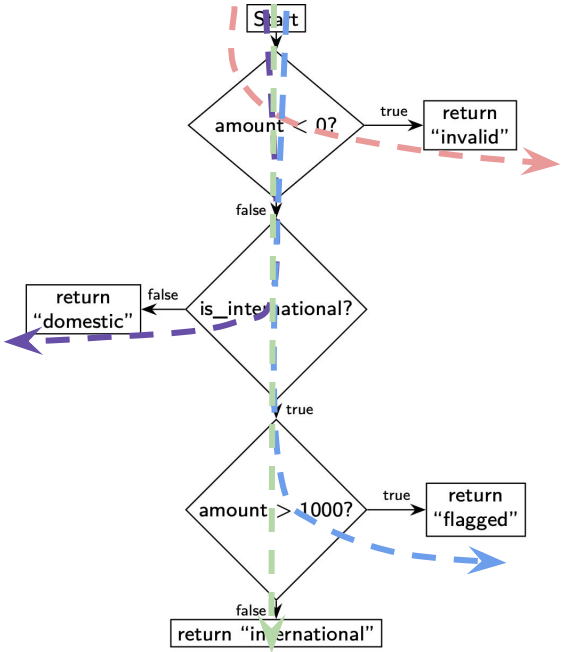
app/payments/transactions.py

```
def classify_transaction(amount,
     is_international):
    if amount < 0:
        return "invalid"
    if is_international:
        if amount > 1000:
            return "flagged"
        else:
            return "international"
    else:
        return "domestic"
```

Nb Statements: 7

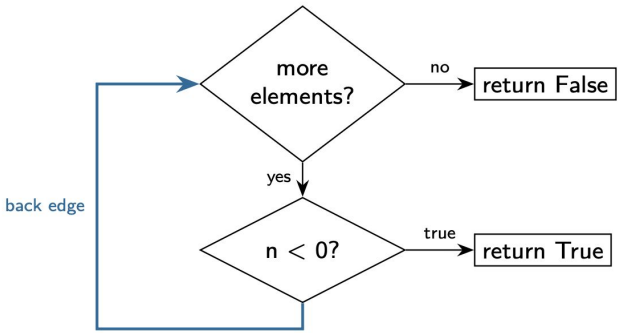
Nb Branches : 2 + 2 + 2

Nb Paths: 4



Control-Flow Graph

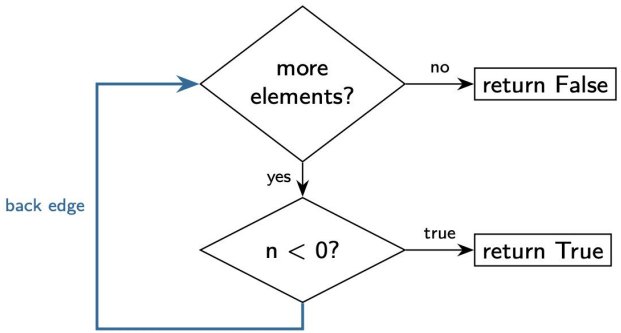
```
def contains_negative(numbers):
    for n in numbers:
        if n < 0:
            return True
    return False
```



Nb Statements: ***4***

Control-Flow Graph

```
def contains_negative(numbers):
    for n in numbers:
        if n < 0:
            return True
    return False
```

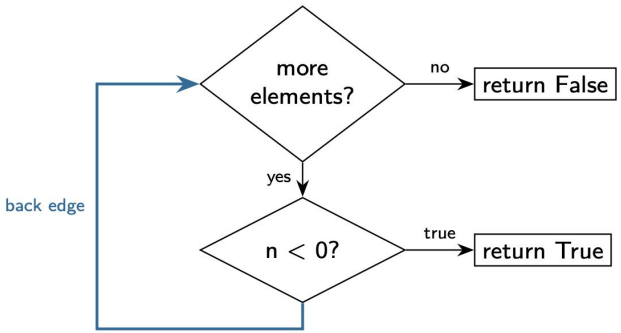


*Nb Statements: **4***

*Nb Branches : **2 + 2***

Control-Flow Graph

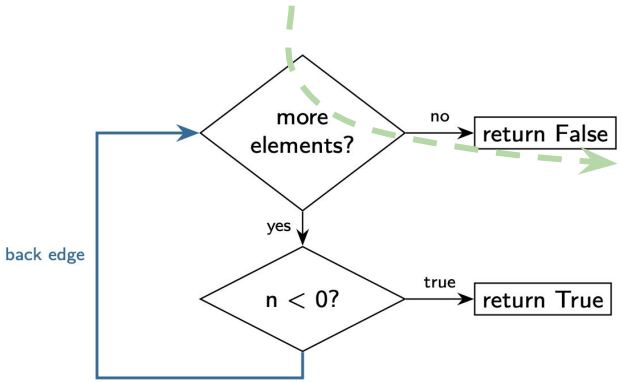
```
def contains_negative(numbers):
    for n in numbers:
        if n < 0:
            return True
    return False
```



*Nb Statements: **4***
*Nb Branches : **2 + 2***
*Nb Paths: **??***

Control-Flow Graph

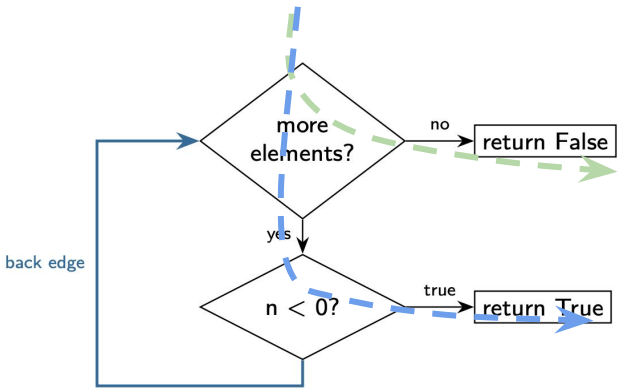
```
def contains_negative(numbers):
    for n in numbers:
        if n < 0:
            return True
    return False
```



*Nb Statements: **4***
*Nb Branches : **2 + 2***
*Nb Paths: **??***

Control-Flow Graph

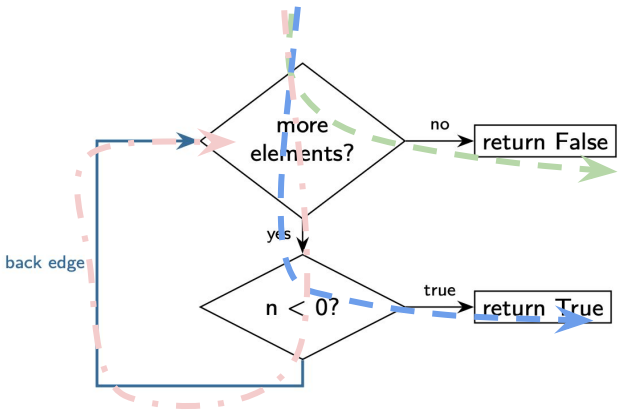
```
def contains_negative(numbers):
    for n in numbers:
        if n < 0:
            return True
    return False
```



*Nb Statements: **4***
*Nb Branches : **2 + 2***
*Nb Paths: **??***

Control-Flow Graph

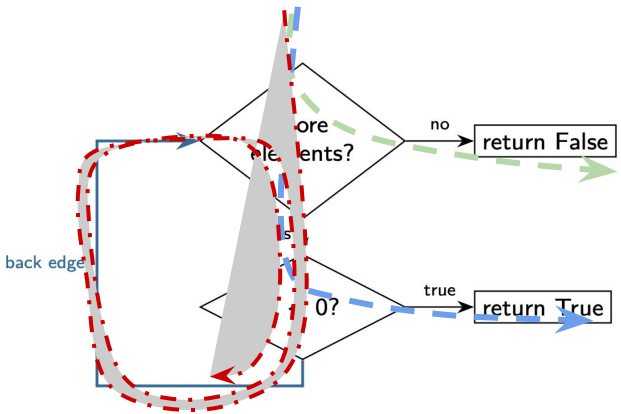
```
def contains_negative(numbers):
    for n in numbers:
        if n < 0:
            return True
    return False
```



*Nb Statements: **4***
*Nb Branches : **2 + 2***
Nb Paths: 3 ???

Control-Flow Graph

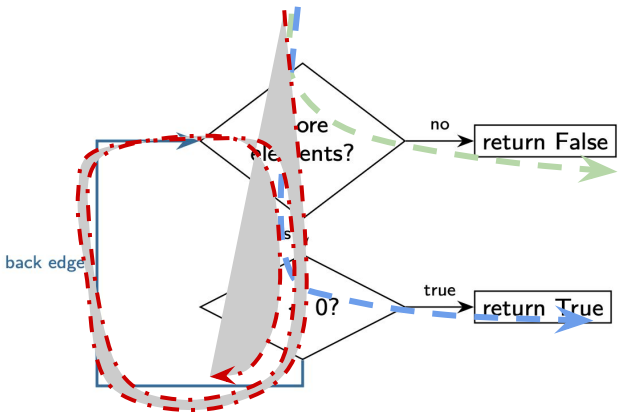
```
def contains_negative(numbers):
    for n in numbers:
        if n < 0:
            return True
    return False
```



Nb Statements: 4
Nb Branches : 2 + 2
Nb Paths: 3, 3+ ???

Control-Flow Graph

```
def contains_negative(numbers):
    for n in numbers:
        if n < 0:
            return True
    return False
```



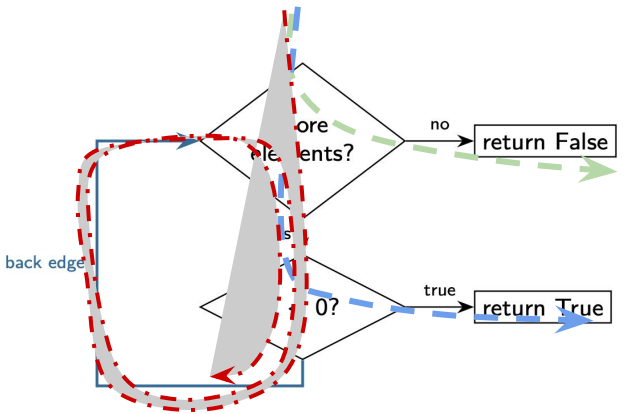
*Nb Statements: **4***

*Nb Branches : **2 + 2***

*Nb Paths: 3, **3+ (actually 2^3 paths;** depends on the input parameter….)*

Control-Flow Graph

```
def contains_negative(numbers):
    for n in numbers:
        if n < 0:
            return True
    return False
```



Basis path testing ($V(G) = 3$) + "zero, one, many" — the repo's test ids:

[] → False	zero-iterations
[-1] → True	one-iteration-early-return
[1] → False	one-iteration-loop-exits
[3,7,-2,9] → True	many-iterations-negative-late

Nb Statements: 4

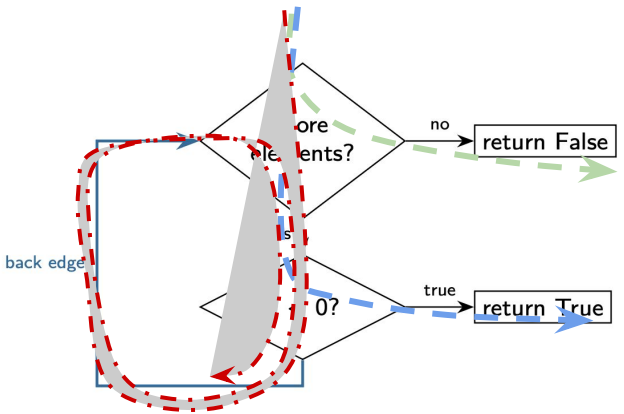
Nb Branches : 2 + 2

*Nb Paths: 3, **3+ (actually 2^3 paths;** depends on the input parameter....)*

Generalize into 3 Basis Path, V(G) = 3; hence 3 tests cases, excluding the [] case

Control-Flow Graph

```
def contains_negative(numbers):
    for n in numbers:
        if n < 0:
            return True
    return False
```



Basis path testing ($V(G) = 3$) + "zero, one, many" — the repo's test ids:

[] → False	zero-iterations
[-1] → True	one-iteration-early-return
[1] → False	one-iteration-loop-exits
[3,7,-2,9] → True	many-iterations-negative-late

Nb Statements: 4

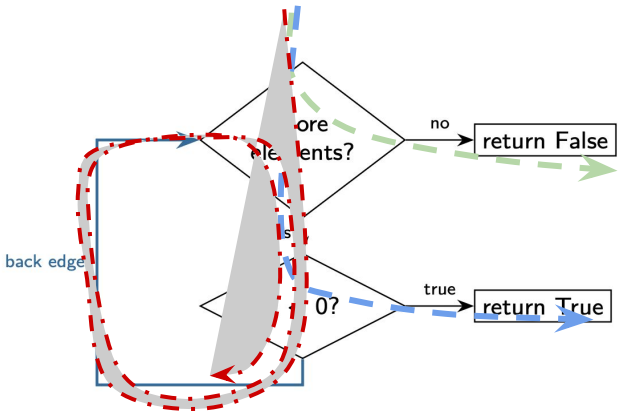
Nb Branches : 2 + 2

*Nb Paths: 3, **3+ (actually 2^3 paths;** depends on the input parameter....)*

Generalize into 3 Basis Path, V(G) = 3; hence 3 tests cases, excluding the [] case

Control-Flow Graph

```
def contains_negative(numbers):
    for n in numbers:
        if n < 0:
            return True
    return False
```



Basis path testing ($V(G) = 3$) + "zero, one, many" — the repo's test ids:

[] → False	zero-iterations
[-1] → True	one-iteration-early-return
[1] → False	one-iteration-loop-exits
[3,7,-2,9] → True	many-iterations-negative-late

Nb Statements: 4

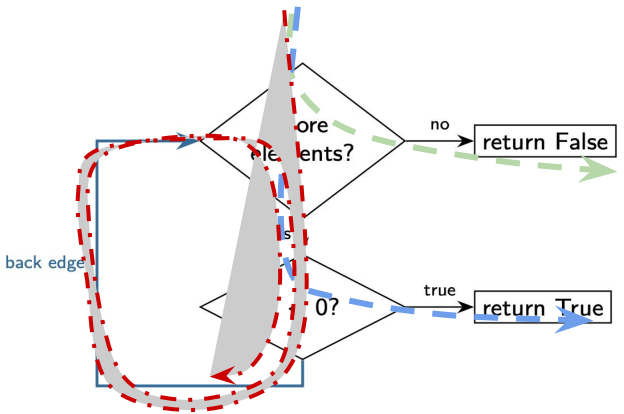
Nb Branches : 2 + 2

*Nb Paths: 3, **3+ (actually 2^3 paths;** depends on the input parameter....)*

Generalize into 3 Basis Path, V(G) = 3; hence 3 tests cases, excluding the [] case

Control-Flow Graph

```
def contains_negative(numbers):
    for n in numbers:
        if n < 0:
            return True
    return False
```



Basis path testing ($V(G) = 3$) + "zero, one, many" — the repo's test ids:

[] → False	zero-iterations
[-1] → True	one-iteration-early-return
[1] → False	one-iteration-loop-exits
[3,7,-2,9] → True	many-iterations-negative-late

Nb Statements: 4

Nb Branches : 2 + 2

*Nb Paths: 3, **3+ (actually 2^3 paths;** depends on the input parameter....)*

Generalize into 3 Basis Path, V(G) = 3; hence 3 tests cases, excluding the [] case



Data Flow Testing

Data Flow Testing

```python
def calculate_price(price, employee):
    discount = 0

    if employee:
        discount = price * 0.10

    discount = price * 0.05

    final_price = price - discount

    return final_price
```

```python
calculate_price(100, True)
```

Suppose a store gives an employee a **10% discount**, and others **5%**.

Can you identify a Bug in the following code?

Data Flow Testing

We can track the

- ***definition (D),*** *and*
- ***use (U)***

```
def calculate_price(price, employee):
    discount = 0                    # D1

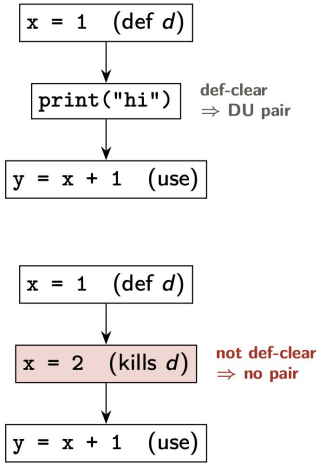
    if employee:
        discount = price * 0.10     # D2

    discount = price * 0.05         # D3  ← redefinition/overwrite

    final_price = price - discount  # U

    return final_price # U
```

Def-Clear and DU Pairs



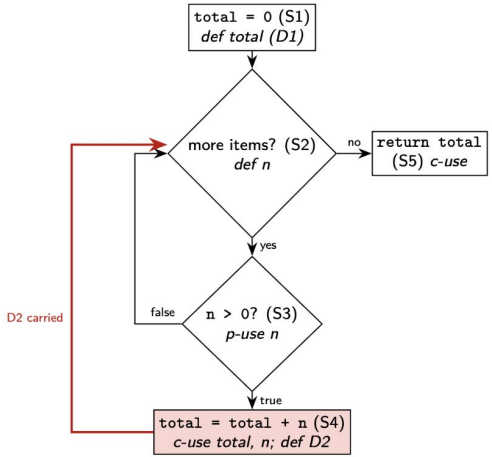
Worked Example: sum_positive()

app/billing/charges.py

```
def sum_positive(numbers):
    total = 0                    # S1
    for n in numbers:            # S2
        if n > 0:                # S3
            total = total + n    # S4
    return total                 # S5
```

Billing reading: charges are positive, credits/refunds are ≤ 0; only charges are summed.

S1	def `total` (**D1**)
S2	def `n`
S3	p-use `n`
S4	c-use `total`, `n`; def `total` (**D2**)
S5	c-use `total`



Worked Example: sum_positive()

#	Var	Def	Use	Notes	Covered by (test id)
1	total	D1 (S1)	S4 (c)	first pass through loop	`all-defs:one-charge`
2	total	D1 (S1)	S5 (c)	S4 never runs	`all-defs:credit-only`
3	total	D2 (S4)	S4 (c)	**loop-carried**	`all-uses:two-charges`
4	total	D2 (S4)	S5 (c)	S4 runs, then loop exits	`all-defs:one-charge`
5	n	S2	S3 true (p)	element > 0	`all-defs:one-charge`
6	n	S2	S3 false (p)	element ≤ 0	`all-defs:credit-only`
7	n	S2	S4 (c)	only if S3 is true	`all-defs:one-charge`

Input	Result	Test id	Criterion reached
`[-1], [3]`	0, 3	`all-defs:*`	All-Defs (and 100% branch)
+ `[3, 5]`	8	`all-uses:two-charges`	**All-Uses** (pair #3)
+ `[3, -2, 5]`	8	`du-path:loop-carried-via-credit`	a second path for #3

Worked Example: sum_positive()

#	Var	Def	Use	Notes	Covered by (test id)
1	total	D1 (S1)	S4 (c)	first pass through loop	`all-defs:one-charge`
2	total	D1 (S1)	S5 (c)	S4 never runs	`all-defs:credit-only`
3	total	D2 (S4)	S4 (c)	**loop-carried**	`all-uses:two-charges`
4	total	D2 (S4)	S5 (c)	S4 runs, then loop exits	`all-defs:one-charge`
5	n	S2	S3 true (p)	element > 0	`all-defs:one-charge`
6	n	S2	S3 false (p)	element ≤ 0	`all-defs:credit-only`
7	n	S2	S4 (c)	only if S3 is true	`all-defs:one-charge`

Input	Result	Test id	Criterion reached
`[-1], [3]`	0, 3	`all-defs:*`	All-Defs (and 100% branch)
+ `[3, 5]`	8	`all-uses:two-charges`	**All-Uses** (pair #3)
+ `[3, -2, 5]`	8	`du-path:loop-carried-via-credit`	a second path for #3

```
pytest tests/billing/test_charges.py -v \
       --cov=app.billing.charges --cov-branch --cov-report=term-missing
```

```
tests/billing/test_charges.py ....                                       [100%]
Name                       Stmts   Miss Branch BrPart  Cover   Missing
----------------------------------------------------------------------------
app/billing/charges.py         6      0      4      0   100%
4 passed in 0.11s
```

Predict Before We Run

Comment out the last two cases (`[3, 5]` and `[3, -2, 5]`). Predict: what happens to **branch coverage**?

```
        pytest.param([-1], 0, id="all-defs:credit-only"),
        pytest.param([3], 3, id="all-defs:one-charge"),
        # pytest.param([3, 5], 8, id="all-uses:two-charges"),
        # pytest.param([3, -2, 5], 8, id="du-path:loop-carried-via-credit"),
```

QA